Cryptography

Diffie-Hellman and ElGamal

Ksawery Możdżyński

Agenda For Today

1. A secret in public
Agree without sending it

2. Diffie-Hellman
Two numbers, one secret

3. Why is it safe?
The discrete logarithm

4. A key, not a cipher
Where DH is used

5. The man in the middle
When nobody checks who is who

6. ElGamal encryption
Diffie-Hellman with a message

7. Signatures and RSA
ElGamal side by side with RSA

Today you will need



Python 3

version 3.8 or newer, nothing to install



An editor

any one you like



Paper and a pen

Task 1 is on paper

The tasks: **GitHub** `Lecture4 / Task2 ... Task7`

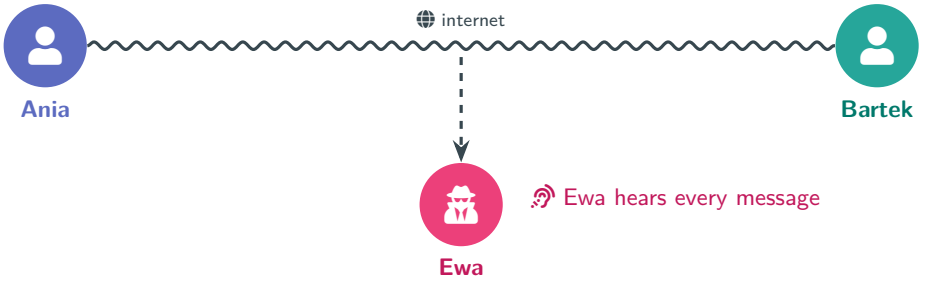
Recall: a symmetric cipher needs a shared key



ChaCha20 and AES encrypt gigabytes per second.
But Ania and Bartek must first have the **same secret key**.

Where does that key come from?

Everybody can hear you



They share no secret yet. Everything they say, Ewa hears too.

Can two people agree on a secret that Ewa hears in full?

Recall: one way from the last lecture

Hybrid encryption with RSA
Ania picks a random key and encrypts it with Bartek's **public key**.
Only Bartek's **private key** opens it.

Today a different way: **nobody sends the key at all**.
Both sides compute it.



last lecture: the key travels,
locked with Bartek's public key



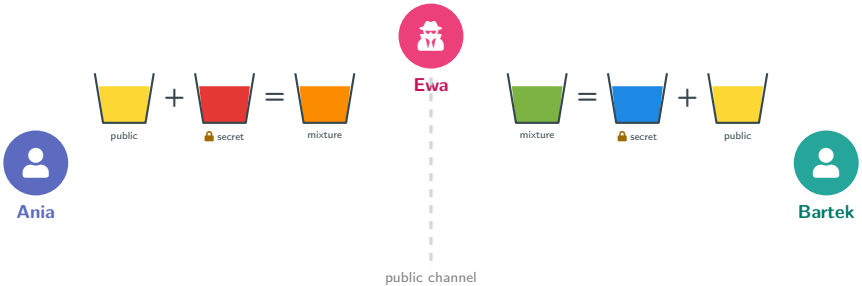
today: the key never travels

Mixing paint, step 1



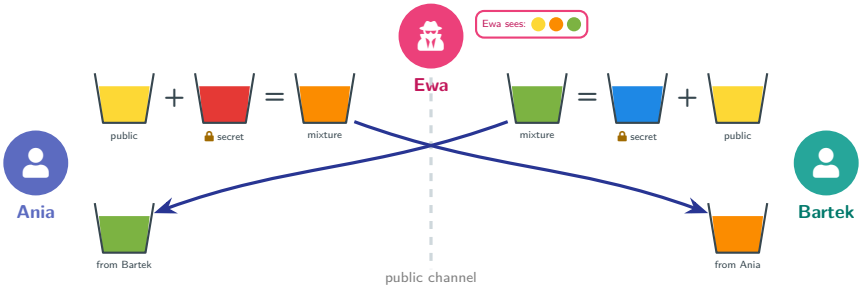
- **1. Both start with a public colour: yellow.**
2. Each adds a secret colour: Ania red, Bartek blue.
3. They swap the mixtures in public.
4. Each adds their own secret colour again.

Mixing paint, step 2



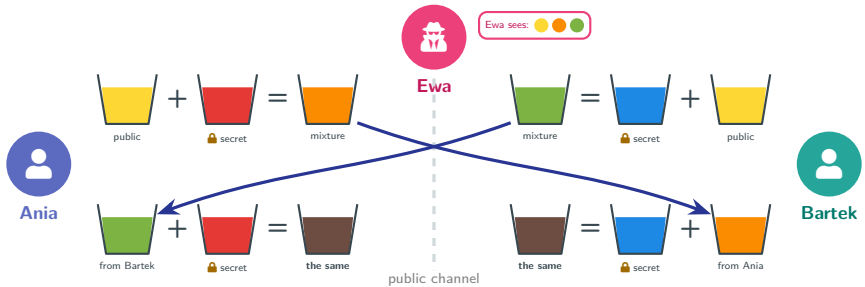
1. Both start with a public colour: yellow.
2. **Each adds a secret colour: Ania red, Bartek blue.**
3. They swap the mixtures in public.
4. Each adds their own secret colour again.

Mixing paint, step 3



1. Both start with a public colour: yellow.
2. Each adds a secret colour: Ania red, Bartek blue.
3. **They swap the mixtures in public.**
4. Each adds their own secret colour again.

Mixing paint, step 4

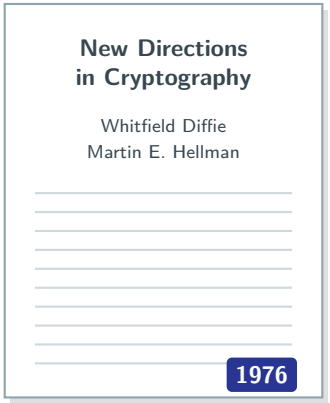


1. Both start with a public colour: yellow.
2. Each adds a secret colour: Ania red, Bartek blue.
3. They swap the mixtures in public.
4. **Each adds their own secret colour again.**

Both now have yellow + red + blue: **the same colour**.

Mixing is easy, unmixing is hard.

Diffie and Hellman, 1976



Whitfield Diffie and Martin Hellman,
"New Directions in Cryptography", 1976.

The first public way to **agree on a secret over an open channel**.

Turing Award, 2015.

Recall: powers modulo p

$$5^3 \bmod 23 = 125 \bmod 23 = 10$$

In Python:

```
$ pow(5, 3, 23)
```

Fast even when every number has **600 digits**.

The public numbers: p and g

public

$p = 23$

a prime

public

$g = 5$

a number smaller than p

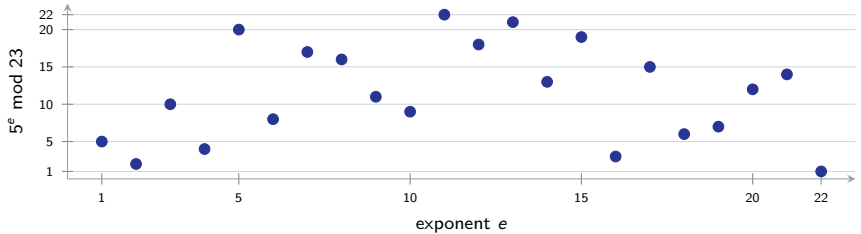
Both are **public**: everybody may know them, Ewa too.

They play the role of the 🟡 **yellow paint**.

Why this g?

Powers of 5 mod 23, for exponents 1 to 22:

exponent	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
$5^e \bmod 23$	5	2	10	4	20	8	17	16	11	9	22	18	21	13	19	3	15	6	7	12	14	1

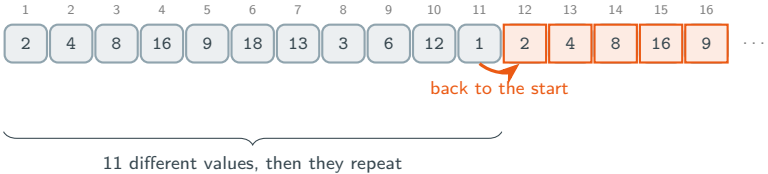


Every number from 1 to 22 appears **exactly once**.

A good g produces every number from 1 to p - 1.

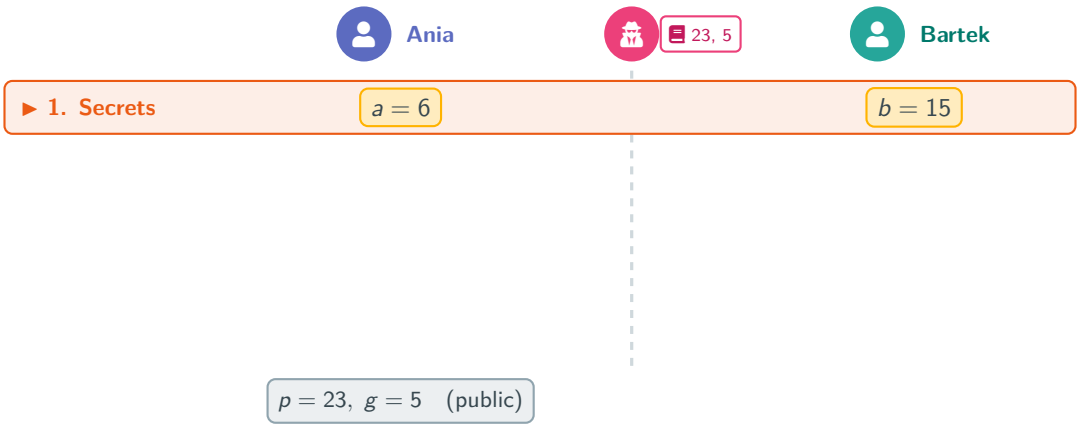
A bad g

Powers of 2 mod 23:

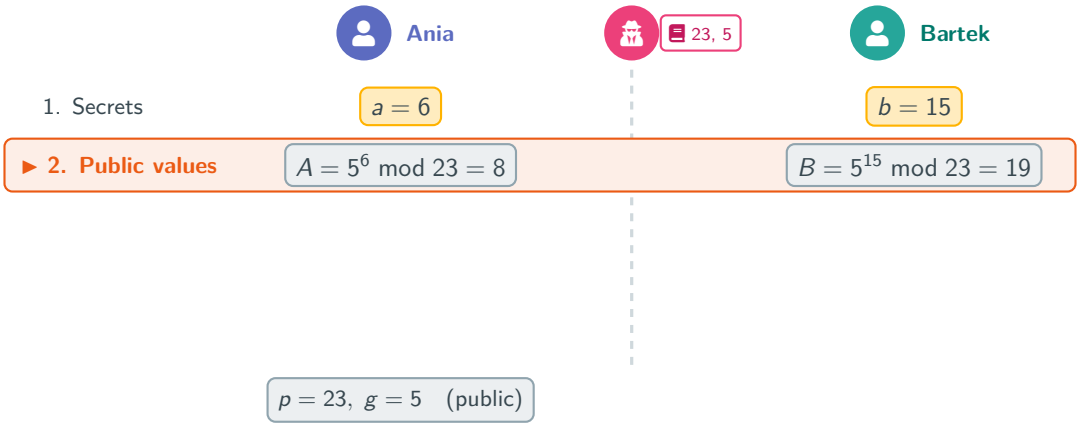


Only **11 different values**: half as many for Ewa to try.

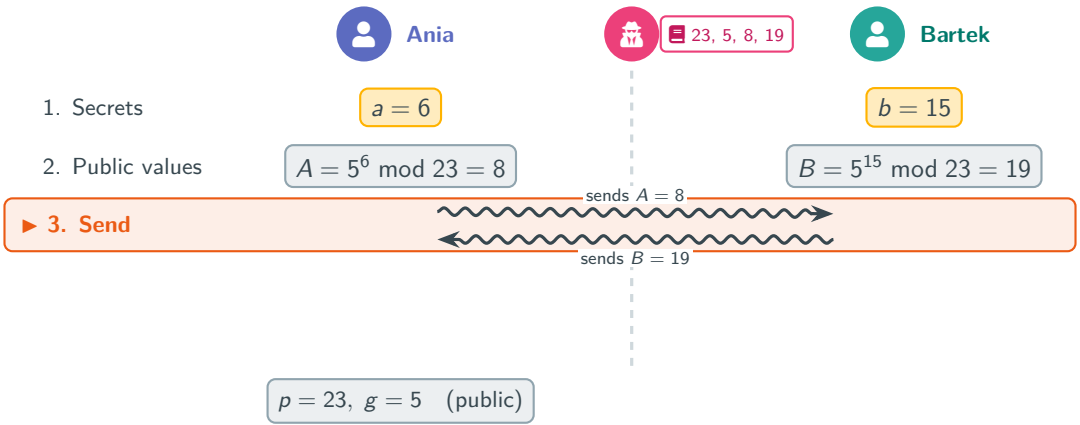
The exchange, step 1



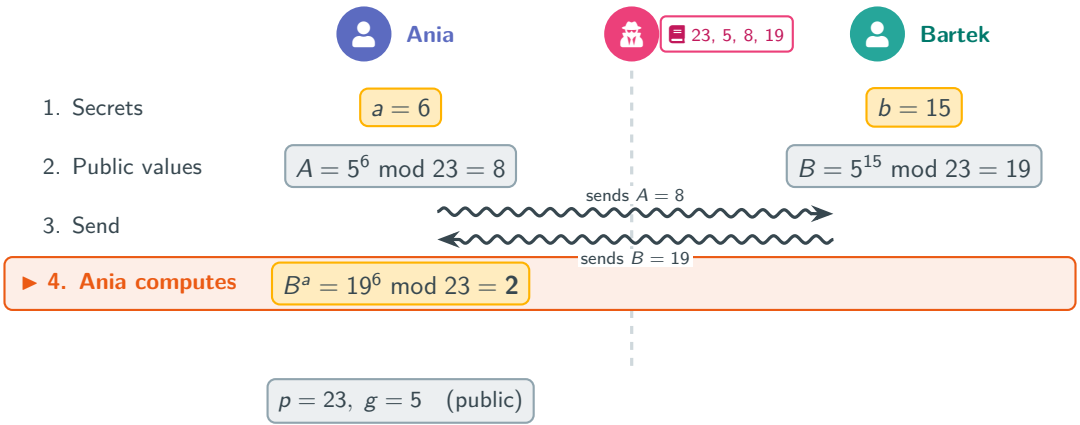
The exchange, step 2



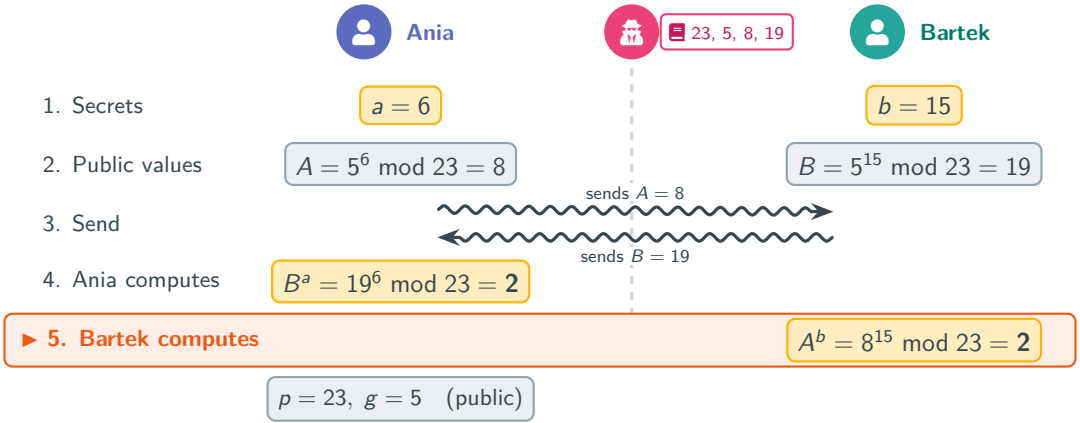
The exchange, step 3



The exchange, step 4



The exchange, step 5

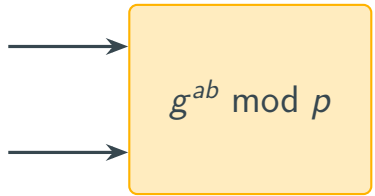


Both have 2. Ewa saw only 23, 5, 8 and 19.

Why are they equal?

Ania: $B^a = (g^b)^a = g^{ba}$

Bartek: $A^b = (g^a)^b = g^{ab}$



$a\,b = b\,a$, so both get $g^{ab} \bmod p$.

Both compute $g^{(ab)} \bmod p$, and nobody ever sent it.

Task 1: an exchange on paper

Task 1

On paper. $p = 23$, $g = 5$, Ania's secret $a = 4$, Bartek's secret $b = 3$

1. Compute A and B.
2. Compute the shared secret as Ania, and again as Bartek.
3. Check that both are the same.

computation	result
A	
B	
Ania: B^a	
Bartek: A^b	

Solution: an exchange on paper

$$A = 5^4 \bmod 23 = 625 \bmod 23 = 4$$

$$B = 5^3 \bmod 23 = 125 \bmod 23 = 10$$

Ania: $B^a = 10^4 \bmod 23 = 10000 \bmod 23 = \mathbf{18}$

Bartek: $A^b = 4^3 \bmod 23 = 64 \bmod 23 = \mathbf{18}$

Real sizes

- p — **2048 bits** (617 digits), a fixed public group, for example group 14 from RFC 3526
- g — 2
- a, b — random numbers of at least 256 bits, **new for every exchange**

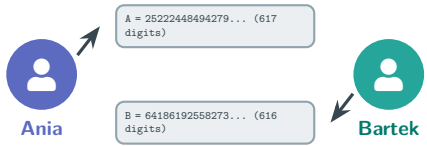
p = 32317006071311007300338913926423828248817941241140239112842009751400741706634354222619689417363569347117901737909704191754605873209195028853758986185622153212175412514901774520270235796078236248884246189477587641105928646099411723245426622522193230540919037680524235519125679715870117001058055877651038861847280257976054903569732561526167081339361799541336476559160368317896729073178384589680639671900977202194168647225871031411336429319536193471636533209717077448227988588565369208645296636077250268955550592836275112117409697299806841055435958486658329164213621823107899099944865246826241697203591185250704536109055 9

Task 2: a real exchange

Task 2

Use the real 2048-bit p and $g = 2$ from RFC 3526. Starter file: `dh.py` from **GitHub**.

1. Pick random secrets a and b.
2. Compute A, B and the secret on both sides.
3. Check that both secrets are equal, and measure the time.



Task 2: what to do

```python
import secrets
P = int("FFFFFFFFFFFFFFFFC90FDAA2...", 16)   # 2048 bits
G = 2

# TODO 1: a and b: secrets.randbits(256)
# TODO 2: A = G^a mod P,  B = G^b mod P
# TODO 3: B^a mod P and A^b mod P
# TODO 4: print whether they are equal,
#         and the first 20 digits: str(x)[:20]
```

Task 2

Open `dh.py` from **GitHub**. Write the code in place of the `TODO` lines.

Solution: a real exchange

```
a = secrets.randbits(256)
b = secrets.randbits(256)
A = pow(G, a, P)
B = pow(G, b, P)
print(pow(B, a, P) == pow(A, b, P))
```

```
$ equal:  True
```

```
$ took 12 ms
```

(4 powers in plain Python)

Four lines of Python, and two strangers share a 2048-bit secret.

What Ewa knows

Ewa's notebook

$p = 23,\ g = 5$

$A = 8$

$B = 19$

~~$a = ?$~~ ~~$b = ?$~~

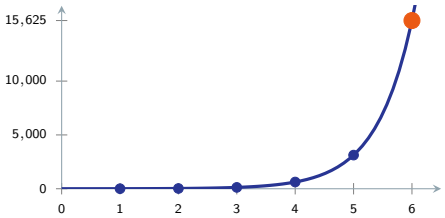
$p,\ g$	public
$A = g^a$	she saw it
$B = g^b$	she saw it

To compute the secret g^{ab}, she needs a **or** b.

The discrete logarithm

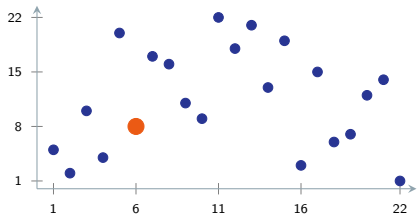
Discrete logarithm: given p, g and $A = g^a \bmod p$, find a.

over normal numbers: 5^a



$5^a = 15625 \rightarrow a = 6$, **easy**: the powers grow steadily

modulo 23: $5^a \bmod 23$



$5^a \bmod 23 = 8 \rightarrow$ the powers **jump around** (5 2 10 4 20 8 ...)

Powers modulo p are easy to compute and hard to undo.

Task 3: find a

Task 3
Four problems: p has 16, 20, 24 and 28 bits. You know p, g and A. Find the secret a by trying $a = 1, 2, 3, \ldots$

16 bits	p = 49223	?
20 bits	p = 939167	?
24 bits	p = 10684799	?
28 bits	p = 187882979	?

Predict first: each p is 4 bits longer. How many times longer will it take?

Task 3: what to do

```python
def find_exponent(p, g, A):
    # TODO: try a = 1, 2, 3, ... : keep value = g^a mod p
    #       (multiply the previous value by g, mod p)
    #       until value == A, then return a
    return None
```

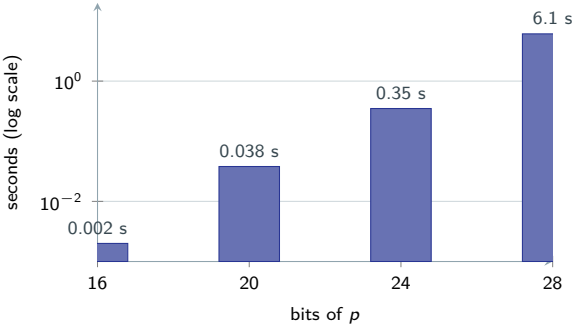
Task 3

Open `dlog.py` from **GitHub**. Write the code in place of the `TODO` lines.

Solution: find a

```
a = 1
value = g
while value != A:
    value = value * g % p
    a += 1
return a
```

16 bits	0.002 s
20 bits	0.038 s
24 bits	0.35 s
28 bits	6.1 s



Every 4 more bits make the search about 16 times longer.

How long for 2048 bits?



Trying every a for a 2048-bit p: about 2^{2047} steps.
Better algorithms exist. For 2048 bits they are still **far too slow**.

Diffie-Hellman is safe as long as the **discrete logarithm is hard**.

When p is too small: Logjam (2015)



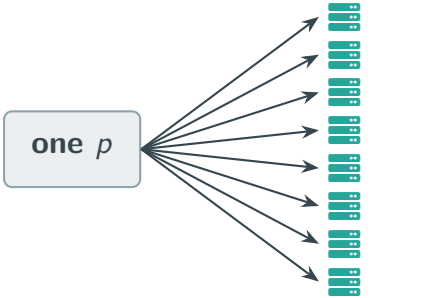
512 bits

Some servers still accepted old 512-bit **“export”** groups in TLS.

The attack
- one week of computing for one fixed p,
- then about one minute for each discrete logarithm in that group

8.4% of the top one million HTTPS sites were vulnerable.

One p for everybody



The week of work depends **only on** p, not on a or b.

A few popular 1024-bit groups were used by millions of servers.

A rich state could do the work **once** and read them all.

Use at least 2048 bits, or the elliptic version from the next lecture.

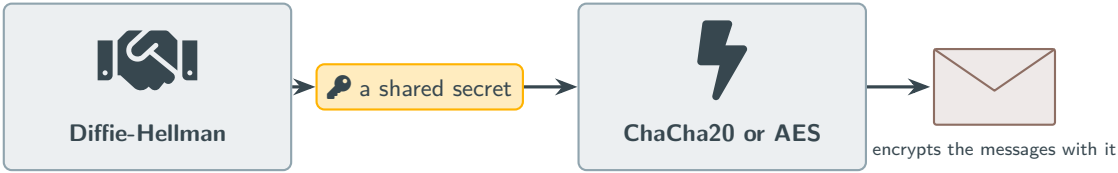
Good to know: the Diffie-Hellman problem



Ewa does not really need a. She needs g^{ab} from g^a and g^b.
Nobody knows a way to do that faster than computing a discrete logarithm.

DH does not encrypt anything

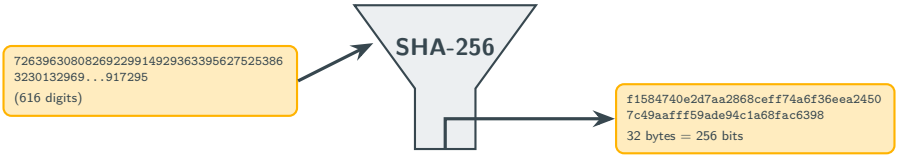
The result of Diffie-Hellman is **one number** that both sides know.



Diffie-Hellman agrees on a key. A symmetric cipher does the rest.

From a number to a key

The secret has **2048 bits**. ChaCha20 needs **256**.

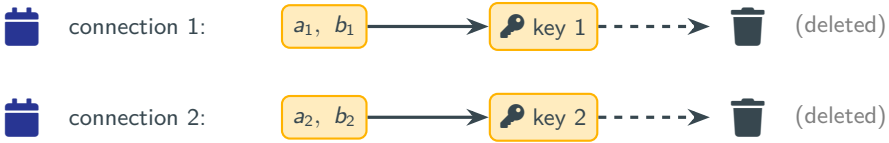



```
key = SHA-256(secret)
```

A **hash function** squeezes the secret into a key of the right size.

A new exchange for every session

Ephemeral Diffie-Hellman: a and b are new random numbers for every connection, and they are deleted when the connection ends.



Forward secrecy

Forward secrecy: a key stolen today does not open conversations recorded yesterday.

RSA sends the key:



steal the private key, decrypt every recorded session

ephemeral Diffie-Hellman:



the session keys are gone, there is nothing to steal

Ephemeral keys protect the past.

Where Diffie-Hellman is used



TLS 1.3 every connection starts with an ephemeral exchange



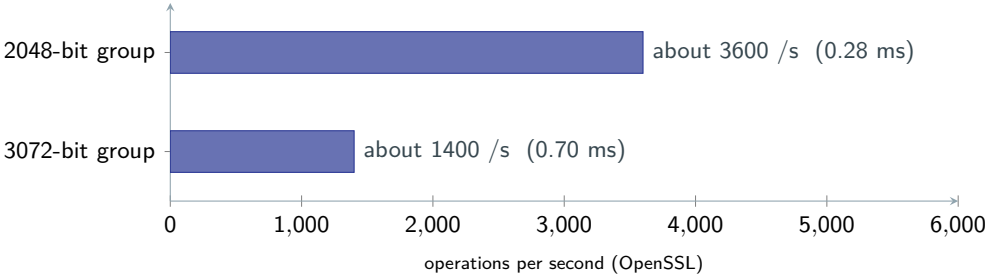
SSH the session key is agreed with Diffie-Hellman



IPsec IKE, the key exchange for many VPNs

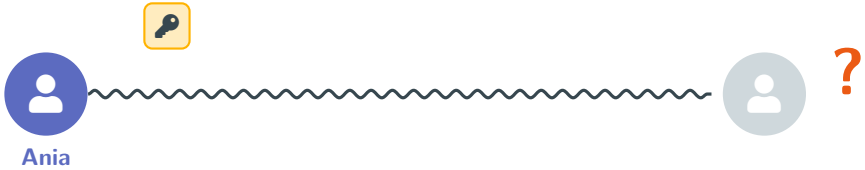
Today mostly in its **elliptic-curve** version: next lecture.

How fast is Diffie-Hellman?



Once per connection, this costs **almost nothing**.

But who is on the other side?



Ania computed a secret **with somebody**.

How does she know it was Bartek?

Mallory can change messages



Ewa listens

Mallory sits on the cable: catches every message and sends what she wants

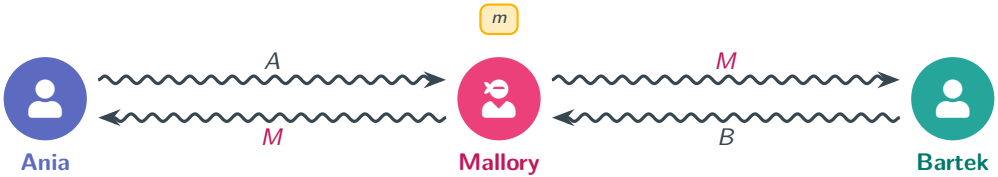
Like a router or a public Wi-Fi under her control.

The attack, step 1



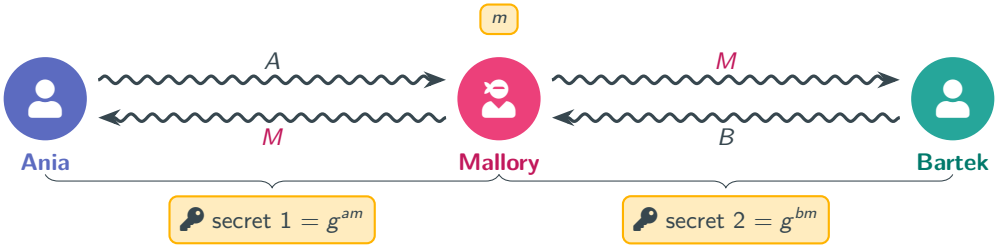
- **1. Ania sends A. Mallory keeps it and sends her own M to Bartek.**
2. Bartek sends B. Mallory keeps it and sends M to Ania.
3. Ania shares a secret with Mallory. Bartek shares another one with Mallory.

The attack, step 2



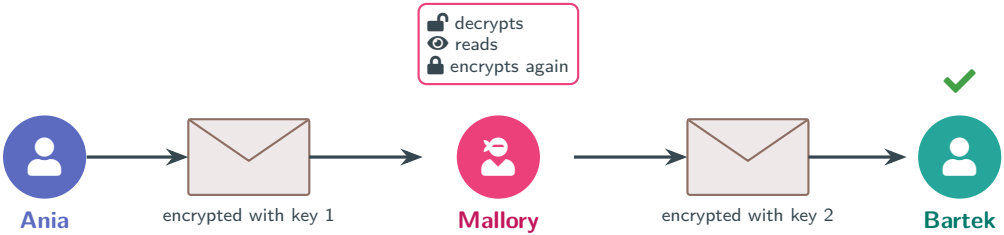
1. Ania sends A. Mallory keeps it and sends her own M to Bartek.
2. **Bartek sends B. Mallory keeps it and sends M to Ania.**
3. Ania shares a secret with Mallory. Bartek shares another one with Mallory.

The attack, step 3



1. Ania sends A. Mallory keeps it and sends her own M to Bartek.
2. Bartek sends B. Mallory keeps it and sends M to Ania.
3. **Ania shares a secret with Mallory. Bartek shares another one with Mallory.**

What the attack gives



Both sides see **correct messages**. Nothing looks wrong.

Diffie-Hellman alone cannot tell who is on the other side.

Task 4: be Mallory

Task 4

A simulated network: Ania and Bartek do a Diffie-Hellman exchange, then Ania sends one encrypted message. **Every value goes through you.**

1. Replace Ania's A and Bartek's B with your own value.
2. Read Ania's message.
3. Pass it on, so that Bartek notices nothing.



Task 4: what to do

```python
class Mallory:
    def __init__(self):
        self.m = secrets.randbits(256)
        self.M = pow(G, self.m, P)

    def change_A(self, A):
        # TODO 1: keep the key shared with Ania, send M instead
        return A

    def change_B(self, B):
        # TODO 2: the same for Bartek
        return B

    def change_message(self, data):
        # TODO 3: decrypt, print, encrypt again for Bartek
        return data
```

Task 4

Open `mitm.py` from **GitHub**. Write the code in place of the `TODO` lines.

Ready in network.py

`key_from_secret`, `encrypt`, `decrypt`

Solution: be Mallory

```
def change_A(self, A):
    self.key_ania = key_from_secret(pow(A, self.m, P))
    return self.M

def change_message(self, data):
    text = decrypt(self.key_ania, data)
    print("Mallory reads:", text)
    return encrypt(self.key_bartek, text)
```

```
$ Mallory reads:  My new card PIN is 7314.  Please keep it secret.
```

```
$ Bartek reads:  My new card PIN is 7314.  Please keep it secret.
```

The fix: authenticate the exchange



signed by Bartek



Mallory has no seal of Bartek

Bartek **signs** his B with his private key.
Ania checks the signature with Bartek's public key.

Mallory can replace B, but she **cannot sign** her M as Bartek.

How does Ania know Bartek's public key?

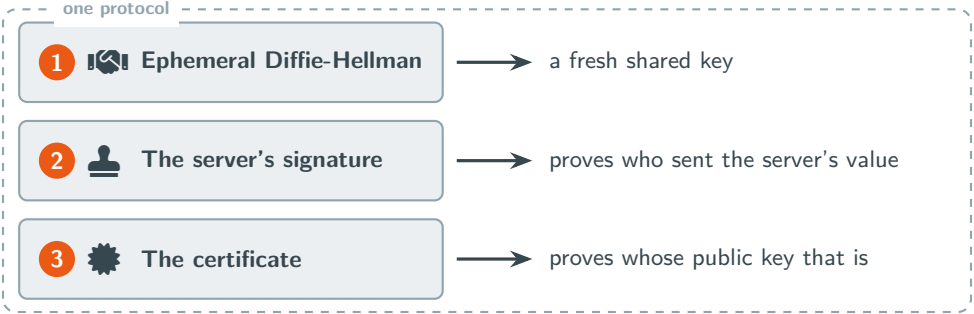


A **certificate**: Bartek's public key, signed by a party that Ania already trusts.

Browsers come with a list of trusted parties.

Diffie-Hellman needs authentication. Without it, it talks to anyone.

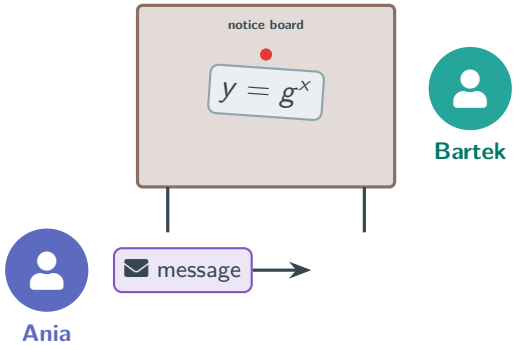
In TLS 1.3


one protocol
1
Ephemeral Diffie-Hellman
a fresh shared key
2
The server's signature
proves who sent the server's value
3
The certificate
proves whose public key that is

Diffie-Hellman as encryption

Taher ElGamal, 1985:

- Bartek publishes his half of the exchange, $y = g^x$, **once and for all**.
- For every message, Ania does her half **on the spot** and uses the shared value to hide the message.



The keys



private key



public key

private key	x	$x = 6$
public key	$y = g^x \bmod p$	$y = 5^6 \bmod 23 = 8$
public too	p and g	$p = 23,\ g = 5$

Encrypt

1. pick a random secret k — $k = 3$
2. $C_1 = g^k \bmod p$ — $C_1 = 5^3 \bmod 23 = 10$
3. $C_2 = M \times y^k \bmod p$ — $M = 10$: $C_2 = 10 \times 8^3 \bmod 23 = 14$

The ciphertext is the pair $(C_1, C_2) = (10, 14)$

Recall: the modular inverse

the inverse of s mod p: the number s^{-1} with $s \times s^{-1} \bmod p = 1$

the inverse of 6 mod 23 is **4**, because $6 \times 4 = 24 = 23 + \mathbf{1}$

In Python:

```
$ pow(6, -1, 23)
```

Decrypt

1. $s = C_1^x \bmod p$ — $s = 10^6 \bmod 23 = 6$
2. $s^{-1} \bmod p$ — $6^{-1} \bmod 23 = 4$
3. $M = C_2 \times s^{-1} \bmod p$ — $M = 14 \times 4 \bmod 23 = 56 \bmod 23 = 10$

The message is back: $M = 10$

Why does it work?

Bartek computes $C_1^x = (g^k)^x = g^{kx}$

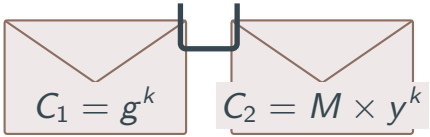
Ania used $y^k = (g^x)^k = g^{xk}$

$g^{kx} \bmod p$

The same value: a Diffie-Hellman secret between k and x.

Ania multiplies by the shared secret. Bartek divides by it.

Why two parts?



$C_1 = g^k$ Ania's half of the exchange: Bartek needs it to compute the secret

$C_2 = M \times y^k$ the message, hidden by the secret

Without C_1, Bartek cannot find y^k. Without C_2, there is no message.

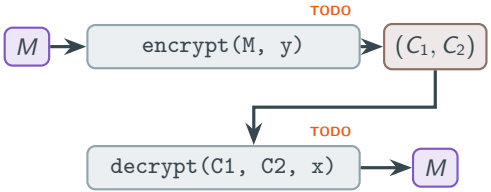
C1 is Ania's half of a Diffie-Hellman exchange. C2 is the message.

Task 5: ElGamal in Python

Task 5
The real 2048-bit group, Bartek's keys are ready.

1. Write `encrypt(M, y)`: a new random k, then C_1 and C_2.
2. Write `decrypt(C1, C2, x)`.

The program encrypts one message twice and decrypts it.



Task 5: what to do

```
def encrypt(M, y):
    # TODO 1: k = secrets.randbelow(P - 2) + 1
    #         return (G^k mod P,  M * y^k mod P)
    return None

def decrypt(C1, C2, x):
    # TODO 2: s = C1^x mod P,  M = C2 * s^-1 mod P
    return None
```

Task 5

Open `elgamal.py` from **GitHub**. Write the code in place of the `TODO` lines.

Solution: ElGamal in Python

```python
def encrypt(M, y):
    k = secrets.randbelow(P - 2) + 1
    return pow(G, k, P), M * pow(y, k, P) % P

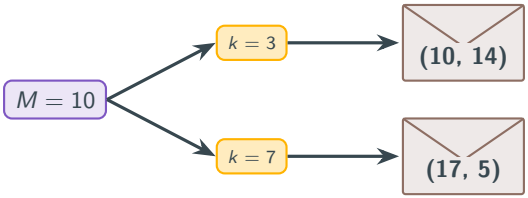
def decrypt(C1, C2, x):
    s = pow(C1, x, P)
    return C2 * pow(s, -1, P) % P
```

```
$ decrypted:  Meet me at the old bridge at noon.
```

```
$ the same message again gives the same C1:  False
```

The same message twice

A new k gives a **new ciphertext**, every time.



textbook RSA: the same message, **the same ciphertext** (the PIN attack)

ElGamal: the same message, **a different ciphertext**

The price: the ciphertext is two numbers, **twice as long as** p.

ElGamal is randomised by design.

Task 6: k used twice

Task 6
Two ciphertexts for Bartek, made with **the same** k: their C_1 is the same.

first:	`C1 = 268389893958...`	`C2 = 178206596531...`
second:	`C1 = 268389893958...`	`C2 = 190653831068...`

the same

- You know the first message: it was printed on a public receipt.
- **Recover the second message** without Bartek's private key.

Task 6: what to do

```
C1 = ...          # the same in both
C2_first, C2_second = ...
first = "Payment approved: 250 EUR, order 1187."

# TODO 1: the mask y^k = C2_first * (inverse of M_first) mod P
# TODO 2: M_second = C2_second * (inverse of the mask) mod P
# TODO 3: print it as text
```

Task 6

Open `reuse.py` from **GitHub**. Write the code in place of the `TODO` lines.

Solution: k used twice

```
mask = C2_first * pow(text_to_number(first), -1, P) % P
second = C2_second * pow(mask, -1, P) % P
print(number_to_text(second))
```

⚠ second message: New door code for the server room: 4096.

The rule: a new k for every message



new every time

k must be **random**, **secret** and **new for every message**.

The same lesson as the keystream and the nonce in the stream cipher lectures.

A repeated k gives away the message, just like a repeated keystream.

The ElGamal signature

1. pick a random k with $\gcd(k, p-1) = 1$
2. $r = g^k \bmod p$
3. $s = (m - \mathbf{x}\, r) \times k^{-1} \bmod (p-1)$

The signature is the pair (r, s)

x = the private key

Verify

$$\text{check:} \quad g^m \bmod p \;==\; y^r \times r^s \bmod p$$

example: $p = 23,\ g = 5,\ x = 6,\ y = 8,\ m = 10,\ k = 3$ gives $r = 10,\ s = 20$

$5^{10} \bmod 23 = \mathbf{9}$ $=$ $8^{10} \times 10^{20} \bmod 23 = \mathbf{9}$ ✔ **valid**

Signed with the private key x, checked with the public key y.

Task 7: sign and verify

Task 7
Small numbers, as on the last slide.

1. Write `sign(m, k)`.
2. Write `verify(m, r, s)`.
3. Check the signature for $m = 10$, then for $m = 11$.



Task 7: what to do

```
p, g, x = 23, 5, 6
y = pow(g, x, p)

def sign(m, k):
    # TODO 1: r = g^k mod p,  s = (m - x r) k^-1 mod (p - 1)
    return None

def verify(m, r, s):
    # TODO 2: g^m mod p == y^r * r^s mod p
    return None
```

Task 7

Open `sign.py` from **GitHub**. Write the code in place of the `TODO` lines.

Solution: sign and verify

```python
def sign(m, k):
    r = pow(g, k, p)
    s = (m - x * r) * pow(k, -1, p - 1) % (p - 1)
    return r, s

def verify(m, r, s):
    return pow(g, m, p) == pow(y, r, p) * pow(r, s, p) % p
```

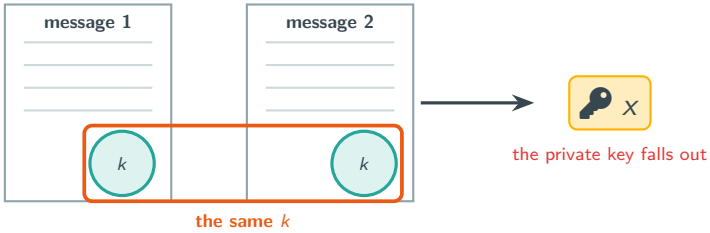
```
$ signature (r, s):  (10, 20)
```

```
$ valid for m = 10:  True
```

```
$ valid for m = 11:  False
```

A changed message no longer matches the signature.

Good to know: k in signatures



A k used for two signatures **gives away the private key** x.

Today's DSA and ECDSA signatures come from the ElGamal signature and have exactly the same weakness. A famous case comes in the next lecture.

ElGamal and RSA side by side

	ElGamal	**RSA**
hard problem	discrete logarithm	factoring
ciphertext	two numbers (C_1, C_2)	one number
randomness	built in (a new k)	needs padding (OAEP)
signatures	(r, s), family of DSA	$S = M^d \bmod n$
⚠ danger	a repeated k	textbook use without padding

Two ways to public-key crypto, each with its own trap.

Where ElGamal lives today



OpenPGP and GnuPG — ElGamal encryption keys; DSA + ElGamal was GnuPG's default for many years



DSA, ECDSA — signatures descended from the ElGamal signature



elliptic curves — the same ideas with shorter keys: next lecture

Summary

1. Diffie-Hellman: both sides compute g^{ab} mod p, nobody sends it.
2. It is safe while the discrete logarithm is hard: p of at least 2048 bits.
3. DH agrees on a key; ephemeral keys give forward secrecy; it needs authentication.
4. ElGamal: $C_1 = g^k$, $C_2 = M\,y^k$; the two parts are a DH exchange and the message; never reuse k.
5. ElGamal signs too; compared with RSA it is randomised but its ciphertext is twice as long.